Project A - Authentication Guide

1. Summary

Project A implemented JWT authentication. Access tokens are signed with the HS256 algorithm using a shared secret held in the secrets manager. Access tokens expire after 15 minutes. Refresh tokens are opaque random strings that expire after 7 days and can be revoked.

2. Login flow

Step 1: the client posts the customer's username and password to the /login endpoint over TLS. Step 2: the authentication service verifies the password hash using a memory-hard algorithm and checks the account is not locked. Step 3: on success the server returns an access token (the JWT) and sets the refresh token in an HttpOnly, Secure, SameSite=Strict cookie. Step 4: the client sends the access token in the Authorization header as a Bearer token for every API call.

Step 5: when the access token expires, the client calls /refresh. The server validates the refresh token, issues a new access token, and rotates the refresh token so that each refresh token can be used only once.

3. Token claims

The JWT payload contains: sub (customer identifier), iat (issued at), exp (expiry), jti (unique token identifier) and scope (list of permitted actions). Personal data such as names or account numbers must never be placed in the token.

4. Secret management and rotation

The HS256 signing secret is at least 256 bits long and is stored in the secrets manager, never in source control or container images. The secret is rotated every 90 days. During rotation the gateway accepts tokens signed with either the current or the previous secret for 20 minutes so that active sessions are not interrupted.

5. Error handling

Invalid credentials return HTTP 401 with a generic message that does not reveal whether the username exists. An expired access token returns 401 with the code token_expired so the client can attempt a refresh. After five failed logins within ten minutes the account is locked for 15 minutes and the event is written to the audit log.

6. Known limitations

HS256 uses a shared secret, so every service that verifies tokens must be trusted with the same secret. The team has noted that moving to an asymmetric algorithm would allow verification without sharing signing capability; this is recorded as a future improvement and is not part of the first release.